Анализ навыков и зарплат Frontend разработчиков

Лабораторная работа 3

Вариант 8 - Программист, разработчик Frontend

Цель и результат

В работе развернуто локальное объектное хранилище RustFS, подготовлен набор вакансий и реализованы два независимых расчета частот навыков, пар и троек: собственный MapReduce и SQL DuckDB. Результаты проверены полностью. Выполнены замеры времени и памяти, зарплатный и региональный анализ.

Основные наблюдения

Показатель	Получено
Вакансий в исходном CSV	229 388
Проверено JSON файлов hh	10 000
Вакансий варианта 8	6 075
Вакансий с навыками	4 571
Вакансий с положительной зарплатой в рублях	2 395
Совпадение полных результатов	36 из 36 запусков
Ошибка задачи маппера и успешный повтор	Проверено, двойного счета нет

Общая медиана опубликованной рублевой зарплаты - 140 000 ₽. В Москве медиана выше, чем в Санкт-Петербурге и Екатеринбурге. Различаются также доля раскрытых зарплат и сочетания навыков.

Формальная проверка охватывает частоты всех навыков и всех сочетаний, а не только первые строки рейтинга. Наблюдаемые зарплатные различия не являются оценкой причинного эффекта навыка.

1 Источники данных и RustFS

CSV содержит 229 388 строк и не содержит повторов vacancy_id. Все 10 000 JSON корректны. Все их идентификаторы уже есть в CSV. JSON сохранены архивом в raw/hh.tar.gz, дубли не добавлялись, поля CSV приоритетны. Для расчета выделены 12 полей без длинных описаний. Общая таблица содержит 229 388 записей и занимает 63 412 458 байт.

Использованы данные вакансий за 2024 год. Полный исходный CSV восстановлен из ранее загруженного объекта it-vacancies/dataset/it_vacancies_2024.csv. Сведения об исходниках сохранены в evidence/data_audit.json.

Использован существующий контейнер hdfs-rustfs-lab, RustFS 1.0.0, S3 127.0.0.1:19000, консоль 127.0.0.1:19001. Бакет hh-lake создан для этой работы. Четыре каталога находятся на одном физическом SSD. Для этого учебного стенда включен RUSTFS_UNSAFE_BYPASS_DISK_CHECK. Независимые физические диски не моделировались.

Подтверждение загрузки данных

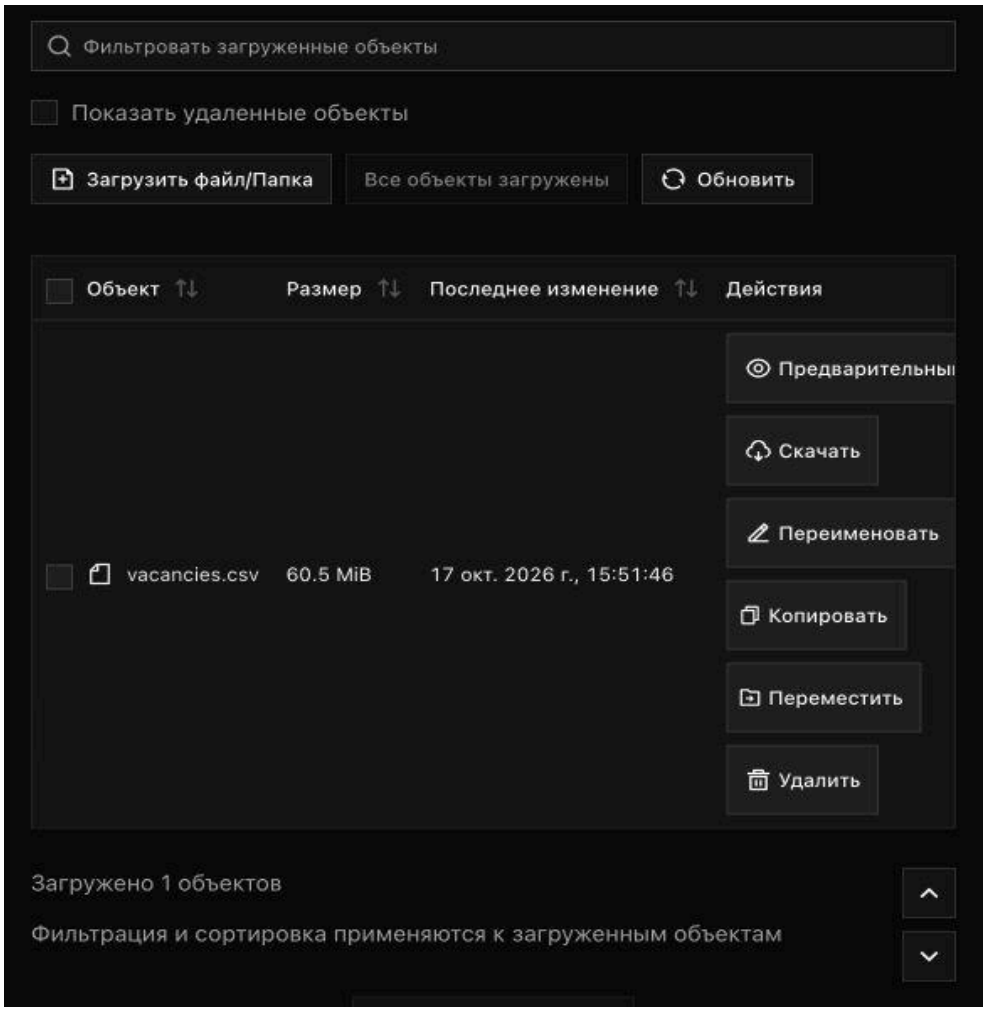


Рисунок 1 - консоль RustFS:

```
$ docker inspect hdfs-rustfs-lab --format "{{.State.Status}} {{.Config.Image}}"
running rustfs/rustfs:1.0.0
$ python src/prepare.py --recover-s3 --hh /path/to/hh
CSV: 229388 JSON: 10000 overlap: 10000 canonical: 229388
```

2 Отбор варианта и независимые вычисления

Вариант 8 определяется по точному значению роли и наличию Frontend в названии. Названия с frontend, front-end, front end, фронтенд и фронтэнд распознаются без учета регистра. Упоминания только в описании не используются.

```
WHERE role = 'Программист, разработчик'
  AND regexp_matches(lower(name),
    'front[\s-]?end|фронт[\s-]?енд|фронт[\s-]?энд')
```

Нормализация навыков: Unicode NFC, нижний регистр, пробелы по краям, множество внутри вакансии. React.js и ReactJS сведены к react, Vue.js и VueJS - к vue, NodeJS - к node.js. Angular и AngularJS, HTML и HTML5, CSS и CSS3 оставлены разными тегами. Пара/тройка - неупорядоченное сочетание разных навыков. Одна вакансия дает каждому сочетанию максимум единицу.

Собственный MapReduce

Координатор читает S3 через потоковый get_object, разбирает CSV с учетом кавычек и формирует чанки по 1000 строк. Два отдельных процесса выполняют Map. В очереди не более четырех чанков. Ключ - отсортированный кортеж из 1, 2 или 3 навыков. Значение - единица. Reduce складывает частоты одинаковых ключей.

Базовый вариант материализует список всех эмиссий чанка. Оптимизированный вариант сразу агрегирует эмиссии в Counter. Он за один потоковый проход сохраняет только строки выбранного сегмента, строит рейтинг и отправляет мапперам чанки этой выборки. Базовый вариант делает два прохода и передает все строки. Буфер сегмента - явный компромисс по памяти.

DuckDB

Расширение httpfs читает тот же s3://hh-lake/variant8/input/vacancies.csv с S3 secret, URL_STYLE=path и USE_SSL=false. Фильтр варианта и нормализация написаны в SQL независимо. UNNEST раскрывает массивы. Условия a<b и b<c исключают перестановки и повторы. GROUP BY считает частоты. DuckDB использует два потока и лимит памяти 1 GB.

```
SELECT [a,b,c] AS skills, 3 AS k, count(*)
FROM stacks, UNNEST(s) AS x(a),
    UNNEST(s) AS y(b), UNNEST(s) AS z(c)
WHERE a < b AND b < c
GROUP BY a,b,c
```

3 Рост числа сочетаний

В эксперименте стек ограничен словарем из 5, 10 и 20 наиболее частых навыков сегмента. Дополнительно посчитан полный словарь. Для каждой вакансии берется пересечение ее навыков с выбранным словарем. Рейтинг независимо строится в Python и SQL. При равенстве частот применяется алфавитный порядок.

Стек	Словарь	Навыки эмиссии	Пары эмиссии	Тройки эмиссии
5	5	10 794	10 439	5 128
10	10	15 795	25 776	25 814
20	20	19 307	41 058	58 679
all	1496	30 336	118 211	382 634
Стек	Уникальные навыки	Уникальные пары	Уникальные тройки	
5	5	10	10	
10	10	45	120	
20	20	190	1 120	
all	1 496	22 643	154 271	

Если после ограничения у вакансии m навыков, она создает m одиночных ключей, C(m,2)=m(m−1)/2 пар и C(m,3)=m(m−1)(m−2)/6 троек. Например, при m=5 это 5, 10, 10. При m=10 - 10, 45, 120. При m=20 - 20, 190, 1140. Общие эмиссии равны сумме этих чисел по вакансиям. Уникальных ключей меньше из-за повторения сочетаний в разных объявлениях.

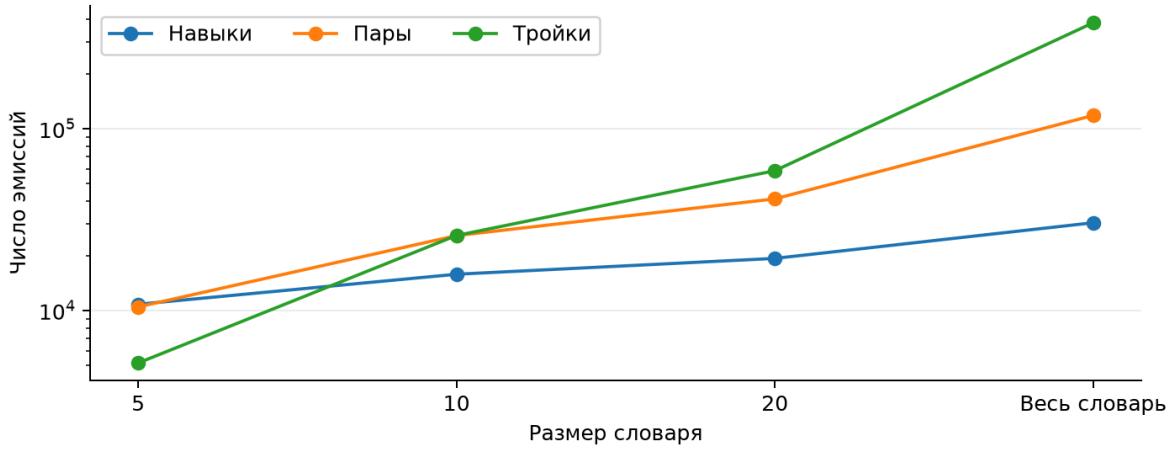


Рисунок 2 - число эмиссий при расширении словаря. Вертикальная шкала логарифмическая

4 Профилирование времени

Для каждого из четырех размеров словаря выполнено по три запуска базового MapReduce, оптимизированного MapReduce и DuckDB: всего 36 новых процессов. Таблица показывает все попытки. Измерение включает импорт библиотек, чтение S3, расчет и сохранение JSON. Подготовка источников и установка зависимостей исключены. Порядок запусков зафиксирован в benchmark.py. Замеры сохранены до переработки оформления кода. После нее отдельно проверено совпадение частот и аналитических таблиц.

Стек	Подход	Попытка 1, с	Попытка 2, с	Попытка 3, с	Медиана, с
5	MR базовый	2,765	2,403	2,345	2,403
5	MR оптим.	1,260	1,195	1,302	1,260
5	DuckDB	1,423	1,354	1,333	1,354
10	MR базовый	2,168	2,066	2,109	2,109
10	MR оптим.	1,145	1,079	1,056	1,079
10	DuckDB	1,136	0,946	0,944	0,946
20	MR базовый	2,142	2,205	2,300	2,205
20	MR оптим.	1,154	1,097	1,055	1,097
20	DuckDB	1,219	1,289	1,113	1,219
all	MR базовый	3,186	3,169	3,440	3,186
all	MR оптим.	2,058	2,030	2,138	2,058
all	DuckDB	2,523	2,145	2,225	2,225

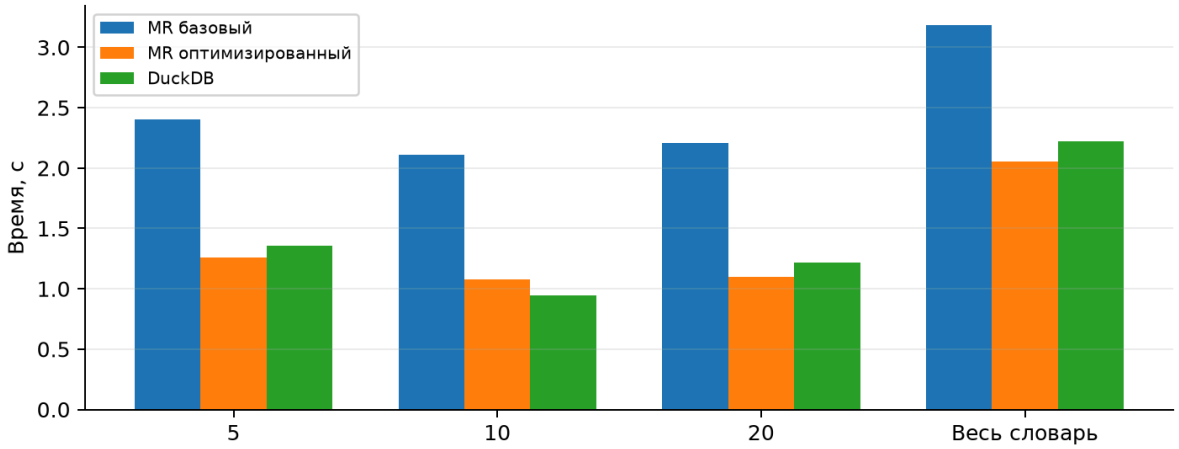


Рисунок 3 - медианное время полного запуска

У базового MapReduce два прохода по объекту, у оптимизированного один с буфером сегмента, у DuckDB один с материализацией входной таблицы. Это часть выбранных алгоритмов. Кэши ОС и RustFS не сбрасывались, расширение httpfs установлено заранее. Поэтому результаты относятся к повторным локальным чтениям. Холодный старт удаленного S3 и пропускная способность сети отдельно не измерялись.

5 Профилирование памяти и оптимизация

Память измерена psutil каждые 10 мс как сумма RSS координатора и всех дочерних процессов. Для DuckDB - RSS процесса с его потоками. Это наблюдаемый пик сэмплирования, а не точный максимум ОС. Общие страницы процессов могут учитываться несколько раз. Машина: macOS 15.3.1 arm64, 10 логических CPU, 16 GB RAM, Python 3.12.14.

Стек	Подход	Пик 1, MiB	Пик 2, MiB	Пик 3, MiB	Медиана, MiB
5	MR базовый	174,7	186,1	182,5	182,5
5	MR оптим.	163,4	161,0	151,9	161,0
5	DuckDB	341,8	311,4	336,5	336,5
10	MR базовый	179,8	183,9	183,2	183,2
10	MR оптим.	158,7	159,1	159,1	159,1
10	DuckDB	339,4	346,2	351,2	346,2
20	MR базовый	178,9	179,3	178,7	178,9
20	MR оптим.	142,8	162,6	161,4	161,4
20	DuckDB	315,2	319,3	308,4	315,2
all	MR базовый	290,9	308,0	282,8	290,9
all	MR оптим.	298,1	322,2	299,2	299,2
all	DuckDB	510,5	458,5	556,4	510,5

До и после оптимизации

На полном словаре медиана времени изменилась с 3,186 до 2,058 с. Ускорение - 1,55 раза. Медиана пика RSS - с 290,9 до 299,2 MiB, изменение потребления памяти на 2,9% относительно базового варианта.

Оптимизация объединяет ранний отбор варианта, один проход S3, передачу мапперам только выбранных строк и combiner без списка эмиссий. Итоговый словарь Reduce все еще хранится в памяти. Выбранный сегмент тоже буферизуется. На малых словарях накладные расходы могут скрыть выигрыш. Три попытки и фоновые приложения ограничивают точность: небольшие различия нельзя считать устойчивым преимуществом без дополнительных повторов.

Это локальное многопроцессное вычисление с собственной схемой MapReduce.

6 Полная верификация и ошибка маппера

Во всех 36 запусках совпали число вакансий, упорядоченный словарь и все записи (навыки, размер сочетания, частота). Для каждого размера сравнивались MR базовый, MR оптимизированный и DuckDB, включая все три повторения. Порядок строк SQL предварительно нормализован. Контроль только суммы частот не использовался.

```
$ python src/benchmark.py
VERIFIED all 36 runs and mapper retry results uploaded to hh-lake
$ python src/test_contract.py
Ran 4 tests
OK
```

Проверены граничный отбор варианта, удаление дублей и алиасов, равенство независимого SQL на маленьком примере, видимость ошибки маппера и обработка зарплат с одной границей. Полные результаты - verification.json и frequency_5/10/20/all.csv.

Сценарий ошибки

В первой задаче маппера искусственно выбрасывается RuntimeError до возврата результата. Координатор фиксирует ошибку и повторяет ровно тот же чанк один раз. Reduce получает только успешно завершенную попытку. Итог совпал с обычным расчетом для словаря 20. Число вакансий тоже совпало. Это ошибка задачи. Принудительное убийство процесса и падение ProcessPool не проверялись.

```
injected_mapper_errors: 1
observed_mapper_errors: 1
retry_limit: 1
counts_equal: true
vacancies_equal: true
```

Проверка сохранения результатов

Вход и результаты повторно прочитаны из hh-lake командой python src/evidence.py. Входной объект побайтово совпал с data/vacancies.csv. Десять проверенных объектов результатов также совпали с локальными файлами.

Вход: s3://hh-lake/variant8/input/vacancies.csv

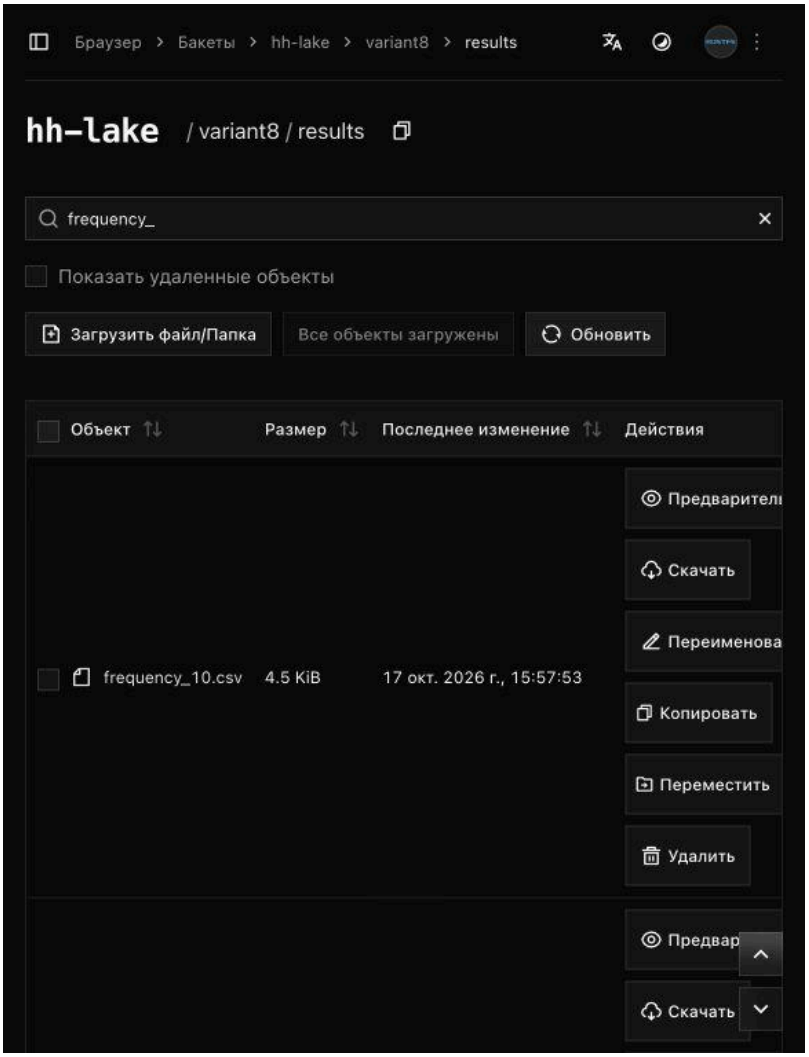
Результаты в s3://hh-lake/variant8/results/

frequency_5.csv - 25 ключей

frequency_10.csv - 175 ключей

frequency_20.csv - 1 330 ключей

frequency_all.csv - 178 410 ключей



Список результатов в консоли RustFS. Полный перечень и размеры объектов приведены в evidence/s3_listing.txt.

7 Зарплаты и пары навыков

Для зарплатного анализа доступны 2 395 объявлений с положительной зарплатой в RUR/RUB из 6 075. Если есть обе границы, используется их середина. Если одна - указанная граница. Число наблюдений: две границы - 1 184, только нижняя - 820, только верхняя - 391. Иностранные валюты не конвертируются. Объявления без зарплат остаются в частотах и региональных знаменателях.

В основную таблицу допускается комбинация с минимум 20 зарплатами и база с минимум 50. База пары - более частый одиночный навык по всем вакансиям сегмента, а не по зарплатной подвыборке. Δ = median(комбинация) − median(база), Δ% = Δ / median(база) × 100. База включает комбинацию. Дополнительная таблица сравнивает с базой без добавленного навыка.

Сочетание и база	N з/п соч./база	Медианы, ₽ соч./база	Δ, ₽	Δ, %
vue | верстка **База: vue**	24/587	300 000 130 000	170 000	130,8
css | разработка по **База: css**	21/600	205 000 105 000	100 000	95,2
javascript | rxjs **База: javascript**	21/1419	230 000 130 000	100 000	76,9
php | react **База: react**	36/909	82 500 150 000	-67 500	-45,0
1с-битрикс | html5 **База: html5**	20/537	67 500 120 000	-52 500	-43,8
jquery | react **База: react**	39/909	85 000 150 000	-65 000	-43,3

Показаны три максимальных положительных и три максимальных отрицательных относительных различия среди прошедших порог. Полный рейтинг - salary_effects.csv. По пять результатов каждого направления - salary_leaders.csv. N относится к зарплатным наблюдениям, а не ко всем вакансиям с сочетанием.

Высокие предложения у сочетаний с Vue, RxJS и тегом «разработка ПО» могут отражать опыт, корпоративные проекты и структуру работодателей. Низкие медианы у сочетаний с PHP и 1С-Битрикс могут указывать на другой класс проектов и уровни позиций. Сам по себе навык не доказан как причина изменения зарплаты.

8 Тройки навыков и устойчивость выводов

Сочетание и база	N з/п соч./база	Медианы, ₽ соч./база	Δ, ₽	Δ, %
javascript | vue | верстка **База: javascript | vue**	23/436	300 000 125 000	175 000	140,0
angular | css | javascript **База: css | javascript**	33/489	180 000 105 000	75 000	71,4
css3 | frontend | адаптивная верстка **База: css3 | адаптивная верстка**	20/82	150 000 90 000	60 000	66,7
git | php | react **База: git | react**	22/394	67 500 140 000	-72 500	-51,8
javascript | php | react **База: javascript | react**	32/657	75 000 140 000	-65 000	-46,4
git | jquery | react **База: git | react**	25/394	80 000 140 000	-60 000	-42,9

Для тройки база - вложенная пара, наиболее часто встречающаяся во всем сегменте. Это соответствует добавлению одного навыка к более распространенной базе. Поля base_vacancies, base_salary_n и base_median_rub доступны в выгрузке. Выбор базы не зависит от величины прибавки.

Сравнение с базой без добавленного навыка

Сочетание	N базы без добавки	Медиана базы без добавки, ₽	Δ к такой базе, ₽
vue | верстка	563	130 000	170 000
php | react	873	150 000	-67 500
javascript | vue | верстка	413	120 000	180 000
git | php | react	372	140 000	-72 500

salary_stratified.csv дополнительно сопоставляет зарплаты внутри одинаковых региона, опыта и признака gross при ≥5 зарплатах сочетания и ≥10 базы в группе. Это диагностическая проверка: малые подгруппы и множественные сравнения не позволяют считать ее доказательством независимого эффекта.

Налоги не пересчитываются. Основная таблица смешивает суммы до и после налогов. Неизвестный налоговый режим и границы вилок ограничивают сопоставимость. Неизвестны фактический доход, бонусы и условия трудоустройства. Самые большие различия после перебора множества сочетаний требуют проверки на новой выборке.

9 Сравнение трех регионов

Выбраны три самых представленных значения area среди всех вакансий варианта: Москва, Санкт-Петербург, Екатеринбург. Это поле региона объявления, оно не гарантирует регион проживания или найма при удаленной работе. В знаменатель входят и вакансии без навыков и зарплаты.

Регион	Всего	С навыками	N з/п	Доля з/п, %	Медиана, ₽
Москва	2775	2004	846	30,5	180 000
Санкт-Петербург	935	702	407	43,5	150 000
Екатеринбург	244	192	134	54,9	115 000

Таблица ниже показывает наиболее частый одиночный навык, пару и тройку в каждом регионе. Частота - число уникальных вакансий, N з/п - число наблюдений для медианы. Полные топ-5 для каждого размера представлены в regions_skills.csv.

Регион и размер	Навыки	Вакансий	N з/п	Медиана, ₽
Москва k=1	javascript	1330	458	180 000
Москва k=2	javascript | typescript	760	221	200 000
Москва k=3	javascript | react | typescript	494	145	200 000
Санкт-Петербург k=1	javascript	485	239	140 000
Санкт-Петербург k=2	javascript | typescript	253	103	150 000
Санкт-Петербург k=3	javascript | react | typescript	153	64	162 500
Екатеринбург k=1	javascript	150	91	120 000
Екатеринбург k=2	javascript | typescript	78	45	140 000
Екатеринбург k=3	css | html | javascript	58	42	105 000

Медиана Москвы (180 000 ₽) выше Санкт-Петербурга (150 000 ₽) на 30 000 ₽, Екатеринбурга (115 000 ₽) - на 65 000 ₽. Это сравнение опубликованных предложений. Состав компаний, опыт и раскрытие зарплат различаются.

10 Наблюдения и ограничения

Навык	Вакансий	Доля всех вакансий варианта, %
javascript	3202	52,7
typescript	2344	38,6
react	2226	36,6
git	1690	27,8
html	1332	21,9
vue	1287	21,2
css	1117	18,4
html5	952	15,7
css3	930	15,3
redux	715	11,8

Наблюдения по регионам

Москва: зарплата раскрыта в 30,5% объявлений. Среднее число нормализованных навыков - 6,34. Самый частый навык javascript присутствует в 47,9% всех объявлений региона. Доля рассчитана от всех вакансий региона.

Санкт-Петербург: зарплата раскрыта в 43,5% объявлений. Среднее число нормализованных навыков - 6,54. Самый частый навык javascript присутствует в 51,9% всех объявлений региона. Доля рассчитана от всех вакансий региона.

Екатеринбург: зарплата раскрыта в 54,9% объявлений. Среднее число нормализованных навыков - 7,11. Самый частый навык javascript присутствует в 61,5% всех объявлений региона. Доля рассчитана от всех вакансий региона.

В частотах общие технологии встречаются чаще нишевых сочетаний, поэтому популярность навыка и медиана зарплаты отвечают на разные вопросы. Для выбора стека полезно одновременно смотреть охват вакансий, зарплатную подвыборку и регион, а не сортировать только по наибольшей прибавке.

11 Воспроизведение и состав сдачи

```
python3 -m venv .venv
.venv/bin/python -m pip install boto3 duckdb psutil matplotlib
.venv/bin/python src/prepare.py --prepared data/vacancies.csv --hh data/hh.tar.gz
.venv/bin/python src/test_contract.py
.venv/bin/python src/analyze.py
.venv/bin/python src/benchmark.py
.venv/bin/python src/evidence.py
```

Команды выполняются из корня проекта при запущенном RustFS на 127.0.0.1:19000. Используются библиотеки boto3, DuckDB, psutil и matplotlib. Для повтора используются подготовленная таблица data/vacancies.csv и архив data/hh.tar.gz. При уже загруженных данных команду prepare.py можно пропустить. README.md указывает вариант и путь к отчету.

Файлы	Назначение
src/*.py, sql/*.sql	Все программы, SQL и проверки
data/vacancies.csv	Точный общий вход из 229 388 строк
results/frequency_*.csv	Полные частоты для 5, 10, 20 и полного словаря
results/profile_runs.csv	36 исходных замеров времени и памяти
results/verification.json	Полная проверка каждого запуска
results/salary_*.csv	Медианы сочетаний, базовые группы и чувствительность
results/regions_*.csv	Региональные зарплаты и навыки всех трех размеров
report/	Отчет